Spark’s Basic Architecture

Apache Spark is a distributed data processing framework. It is designed to handle large datasets by dividing the work across multiple machines and processing them in parallel.

Instead of running everything on a single system, Spark spreads the work across a cluster of computers. This makes processing faster and more efficient.

The main components of Spark architecture are:

- Cluster Manager
- Spark Session
- Spark Context
- Driver Node
- Worker Nodes
- Executors
- Client Mode and Cluster Mode

Each of these components has a specific role, and together they form the complete Spark system.

1. Cluster Manager

The Cluster Manager is responsible for managing the entire cluster and allocating resources.

When a Spark application starts, it does not directly use machines. Instead, it requests resources from the Cluster Manager. The Cluster Manager then decides:

- Which machines (worker nodes) will be used
- How much CPU and memory should be given
- How many executors should run

Common types of cluster managers include:

- Standalone: Spark’s own built-in manager
- YARN: Used in Hadoop environments
- Mesos: General-purpose cluster manager
- Kubernetes: Modern container-based orchestration system

The Cluster Manager ensures that resources are shared properly between multiple applications running at the same time.

2. Spark Session

SparkSession is the entry point for working with Spark.

Whenever you start a Spark application, the first thing you create is a SparkSession. It acts as a single access point to all Spark functionalities.

Through SparkSession, you can:

- Read data from files (CSV, JSON, databases)
- Create DataFrames
- Run SQL queries
- Perform data transformations

It also internally manages SparkContext and other configurations.

In simple terms, SparkSession is the interface between the user and Spark.

3. Spark Context

SparkContext is the core engine that connects your application to the cluster.

It is responsible for:

- Establishing connection with the Cluster Manager
- Sending tasks to worker nodes
- Managing distributed execution

Earlier, developers directly used SparkContext. But now, it is created automatically when you create a SparkSession.

SparkContext works at a lower level compared to SparkSession and mainly deals with RDDs (Resilient Distributed Datasets).

4. Driver Node

The Driver Node is where the main Spark application runs.

It plays a very important role in execution. Its responsibilities include:

- Reading and understanding the user's code
- Dividing the work into smaller tasks
- Creating an execution plan (DAG – Directed Acyclic Graph)
- Sending tasks to worker nodes
- Monitoring execution

- Collecting and combining results

The Driver contains important components like:

- SparkSession
- SparkContext
- DAG Scheduler
- Task Scheduler

The Driver acts as the brain of the entire system because it controls how the work is done.

5. Worker Nodes

Worker Nodes are the machines in the cluster where actual computation happens.

Each worker node:

- Receives tasks from the Driver
- Executes the assigned tasks
- Stores intermediate data in memory or disk

A cluster usually has multiple worker nodes, which allows Spark to process large data in parallel.

The more worker nodes available, the faster the processing can be, depending on the workload.

6. Executors

Executors are processes that run inside worker nodes.

When the Cluster Manager allocates resources, executors are created on worker nodes.

Each executor:

- Executes tasks assigned by the Driver
- Stores data in memory (this is important for caching and performance)
- Sends results back to the Driver

Executors are long-running processes, meaning they stay active for the entire duration of the Spark application.

A single worker node can have multiple executors depending on available CPU and memory.

7. Client Mode and Cluster Mode

Spark applications can run in two different modes depending on where the Driver runs.

Client Mode

In Client Mode:

- The Driver runs on the user's local machine
- Executors run on the cluster

This mode is useful for:

- Development
- Testing
- Debugging

However, it has a limitation:
If the user's system shuts down or loses connection, the job will stop.

Cluster Mode

In Cluster Mode:

- The Driver runs inside the cluster
- The Cluster Manager manages the Driver and Executors

This mode is used in:

- Production environments
- Large-scale data processing

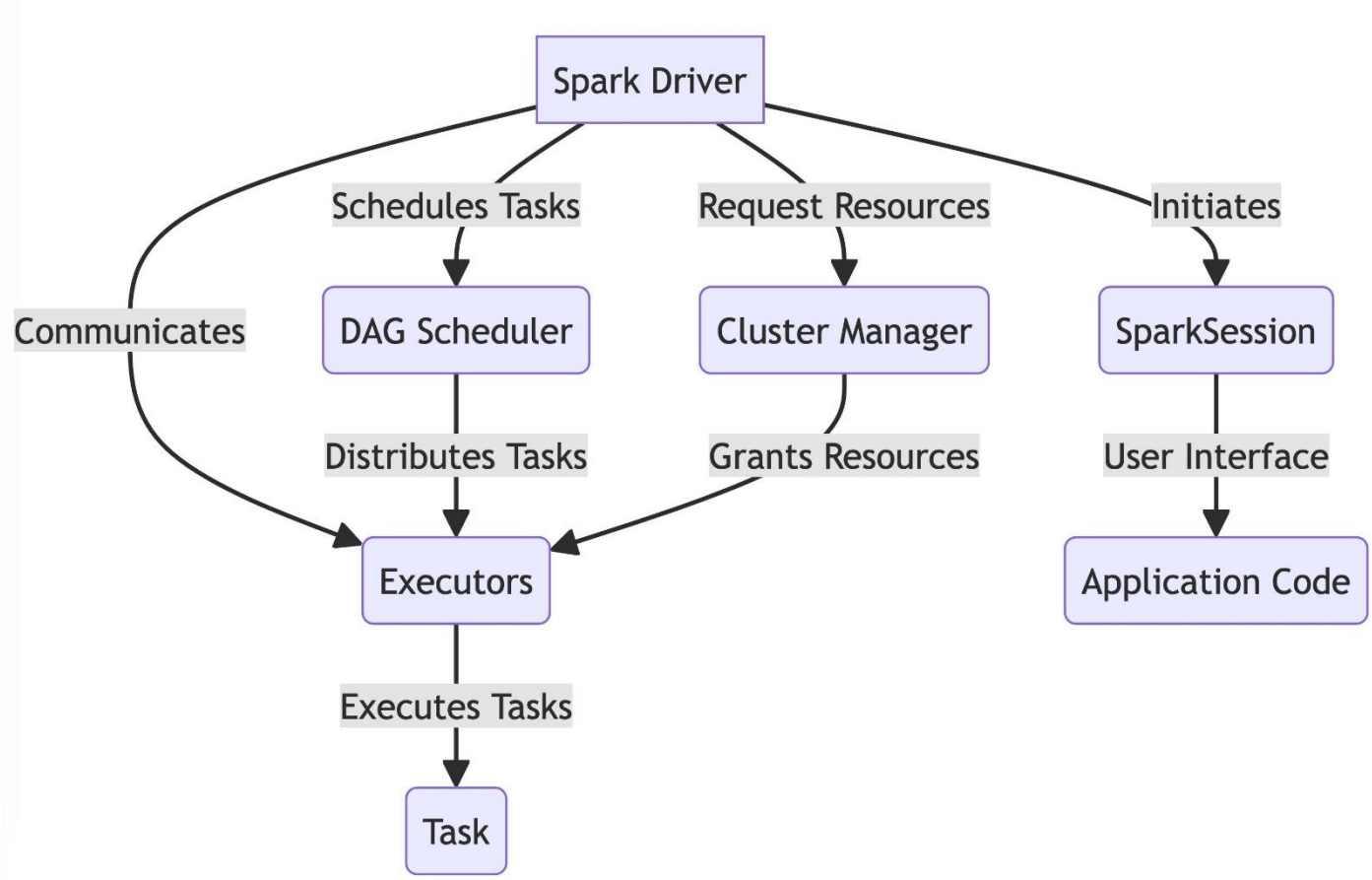
It is more reliable because:

- The job continues even if the user disconnects
- Better fault tolerance

How Everything Works Together

The execution of a Spark job happens in the following steps:

1. The user writes a Spark program
2. A SparkSession is created
3. The Driver Node starts running the application
4. SparkContext connects to the Cluster Manager
5. The Cluster Manager allocates resources

6. Executors are created on Worker Nodes
7. The Driver converts the program into smaller tasks
8. Tasks are distributed to Executors
9. Executors process the data in parallel
10. Results are sent back to the Driver
11. The final output is returned to the user

This process allows Spark to handle large-scale data efficiently.